

Code Complexity Analysis for USEI14

Maximum Throughput (Max Flow)

Project: Logistics On Rails

Sprint: 3

User Story: USEI14 - Calculate Maximum Throughput

Date: January 2026

Overview

This document analyzes the time complexity of all critical methods in the USEI14 implementation using Big-O notation.

Dataset: Belgian Railway Network - $V = 559$ stations (vertices) - $E \approx 1382$ connections (edges - bidirectional graph)

Algorithm:

Edmonds-Karp Algorithm (Implementation of Ford-Fulkerson using BFS) - ESINF06-Graph.pdf, slide 141

Method Analysis

1. EdmondsKarp.bfs()

This method searches for the shortest augmenting path (in number of hops) in the residual graph.

Code Analysis:

```
private boolean bfs(Graph< residualGraph, V source, V sink, Map< parent) {
    // Initialization
    Queue q = new LinkedList<>();    // O(1)
    q.add(source);                    // O(1)
    Set visited = new HashSet<>();    // O(1)
    visited.add(source);              // O(1)

    // BFS Loop
    while (!q.isEmpty()) {            // Loop executes V times in worst case
        V u = q.poll();               // O(1)

        for (Edge edge : residualGraph.outgoingEdges(u)) { // Iterate adjacency list
            // Check capacity > 0 and not visited
            if (!visited.contains(v) && edge.getWeight() > 0) {
                parent.put(v, edge);    // O(1)
                visited.add(v);         // O(1)
            }
        }
    }
}
```

```

        q.add(v);                // O(1)
    }
}
}
return visited.contains(sink);    // O(1)
}

```

Logic: Standard Breadth-First Search (BFS).

- **Vertices:** Each vertex is enqueued and dequeued at most once.
- **Edges:** For each vertex, we iterate over its outgoing edges. In the worst case (connected graph), all edges are inspected.

Complexity: $O(V + E)$

Deterministic: YES

2. EdmondsKarp.computeMaxFlow()

This is the main loop of the algorithm that repeatedly finds augmenting paths and updates flow.

Code Analysis:

```

public Double computeMaxFlow(Graph network, V source, V sink) {
    // 1. Build Residual Graph
    // Copy V vertices and E edges
    // Complexity: O(V + E)

    double maxFlow = 0;

    // 2. Main Loop (Edmonds-Karp)
    // In Edmonds-Karp, the number of augmentations is bounded by O(V * E)
    while (bfs(residualGraph, source, sink, parent)) { // BFS takes O(V + E) ~ O(E)

        double pathFlow = Double.MAX_VALUE;

        // Find bottleneck (backtrack path)
        // Path length is at most V
        for (v = sink; v != source; v = parent.get(v).source) { // O(V)
            pathFlow = Math.min(pathFlow, capacity);
        }

        // Update Residual Graph (Forward and Backward edges)
        for (v = sink; v != source; v = parent.get(v).source) { // O(V)
            // Update forward edge capacity
            // Update backward edge capacity
        }
    }
}

```

```

        maxFlow += pathFlow;
    }

    return maxFlow;
}

```

- **Number of Augmentations:** Ideally, Ford-Fulkerson depends on max flow f^* . However, using BFS (Edmonds-Karp), it is proven that the number of augmentations is at most $O(V \cdot E)$.
- **Cost per Augmentation:** Each BFS takes $O(E)$.
- **Total:** $O(V \cdot E) \times O(E) = O(V \cdot E^2)$.

Overall Complexity: $O(V \cdot E^2)$

Deterministic: YES

3. ComputeMaxFlowController.calculateMaxFlow()

The controller acts as an orchestrator.

Code Analysis:

```

public Double calculateMaxFlow(Station source, Station sink) {
    // Check network
    if (network == null) ...           // O(1)

    long startTime = System.nanoTime();

    // Call Algorithm
    double maxFlow = algorithm.computeMaxFlow(network, source, sink); // O(V * E^2)

    // Logging/Metrics
    long endTime = System.nanoTime(); // O(1)

    return maxFlow;
}

```

Complexity: $O(V \cdot E^2)$ (Dominated by the algorithm)

Deterministic: YES

4. BelgianNetworkLoader.loadNetwork()

Loading the graph from CSV files. Note that for USEI14, the graph is bidirectional (`isBidirectional = true`), effectively doubling the number of edges compared to the CSV lines.

Code Analysis:

```
public static Graph loadNetwork(..., boolean isBidirectional) {
    // Load Stations
    loadStations();           //  $O(V)$  - Read  $V$  lines

    // Initialize Graph
    new MapGraph<>(true); //  $O(1)$ 

    // Load Lines
    while (csv.readLine()) { // Executes  $E_{csv}$  times (physical lines)
        // ... parse ...
        graph.addEdge(u, v); //  $O(1)$ 
        if (isBidirectional) {
            graph.addEdge(v, u); //  $O(1)$ 
        }
    }
}
```

Complexity: $O(V + E)$

Deterministic: YES

Summary Table

Method	Class	Time Complexity	Deterministic
bfs()	EdmondsKarp	$O(V + E)$	YES
computeMaxFlow()	EdmondsKarp	$O(V \cdot E^2)$	YES
calculateMaxFlow()	ComputeMaxFlowController	$O(V \cdot E^2)$	YES
loadNetwork()	BelgianNetworkLoader	$O(V + E)$	YES

Where: - $V = 559$ stations - $E \approx 1382$ connections (edges)

Global Complexity

Overall USEI14 Complexity: $O(V \cdot E^2)$

Breakdown: - **Data Loading:** $O(V + E)$ - **Algorithm Execution:** $O(V \cdot E^2)$

Since $O(V \cdot E^2)$ grows much faster than linear loading time, the global complexity is defined by the Edmonds-Karp algorithm.

Theoretical Note:

For dense graphs where $E \approx V^2$, the complexity could approach $O(V^5)$. However, the railway network is a sparse graph (average degree is low, ≈ 2.5), making

$E \approx k \cdot V$. In this practical scenario, the performance is significantly better than the worst-case upper bound, often behaving closer to $O(V \cdot E)$.